



Cricket Analytics System

A Data-Driven Approach to Match Prediction and Player Performance Analysis

<https://github.com/MuHamzaSheikh/cricket-analytics-data-science>

Name	Enrollment
Muhammad Hamza Sheikh	01-134232-119
Shoaib Malik	01-134232-174

Introduction to Data Sciences
Professor: Dr. Arif-Ur-Rehman

Department of Computer Sciences
BAHRIA UNIVERSITY ISLAMABAD

Contents

1. Abstract.....4

2. Introduction.....5

 2.1 Background and Context5

 2.2 Problem Statement and Motivation5

 2.3 Project Objectives.....5

3. Literature Review6

 3.1 Overview of Cricket Analytics Research6

 3.2 Key Related Studies.....6

 Study 1 Machine Learning for Cricket Match Prediction6

 Study 2 Player Performance Analysis and Role Classification.....6

 Study 3 Feature Engineering for Sports Analytics6

 Study 4 Time Series Analysis in Sports Performance6

 Study 5 Real-Time Analytics and Caching Strategies6

 3.3 Gap in Existing Systems.....6

4. Methodology7

 4.1 Data Collection and Sources7

 4.2 Data Preprocessing and Cleaning7

 Text Normalization.....7

 Numeric Conversion.....7

 Date and Temporal Parsing7

 4.3 Exploratory Data Analysis.....7

 4.4 Feature Engineering.....7

 Player-Level Features7

 Team-Level Aggregated Features8

 Match Context Features8

 4.5 Model Selection Random Forest Classifier8

 4.6 Model Training and Validation Strategy8

5. System Architecture9

 5.1 High-Level Architecture.....9

 5.2 Data Processing Pipeline9

 5.3 Caching Strategy10

 5.4 API Endpoints (FastAPI).....10

 5.5 Frontend Architecture (React).....10

6. Tools and Technologies.....11

 6.1 Backend Stack.....11

 6.2 Frontend Stack11

 6.3 Data Storage and Serialization11

7. Implementation.....	12
7.1 Project Structure.....	12
7.2 Core Analytics Engine	12
7.3 FastAPI Application	12
7.4 React Frontend	12
8. Results and Evaluation	13
8.1 Model Performance Metrics	13
8.2 Precision, Recall, and F1-Score	13
8.3 Feature Importance Analysis	13
8.4 Model Performance by Format	13
8.5 Computational Performance	14
8.6 Data Coverage Summary.....	14
9. Discussion.....	15
9.1 Interpretation of Results	15
9.2 Limitations.....	15
Data Quality	15
Model Constraints	15
10. Conclusion	16
10.1 Summary of Achievements.....	16
10.2 Learning Outcomes.....	16
10.3 Project Impact	16
11. Future Work and Recommendations	17
11.1 Model Enhancement	17
11.2 Feature Engineering Expansion.....	17
11.3 System Scalability	17
11.4 Research Extensions	17
12. References.....	18

1. Abstract

This report presents a comprehensive data science project focused on developing an integrated cricket analytics system that leverages historical international cricket data spanning from 1877 to 2025. The system employs advanced data preprocessing techniques, feature engineering methodologies, and machine learning algorithms to enable accurate match outcome prediction and detailed player performance analysis.

The project implements a full-stack architecture comprising a Python-based FastAPI backend, a React.js frontend, and a robust caching mechanism for performance optimization. The analytics engine processes over 150 years of international cricket data across three primary formats ODI, T20I, and Test generating 20 engineered features that capture player roles, performance metrics, and contextual match information.

A Random Forest Classifier model trained on historical match data achieved 74.6% test accuracy in match outcome prediction. This project demonstrates the practical application of data science methodologies in the sports domain and provides an interactive platform for coaches, analysts, and enthusiasts to make data-driven decisions.

2. Introduction

2.1 Background and Context

Cricket is a sport of significant global relevance, with international matches involving nations from across the world competing in standardized formats. The accumulation of cricket statistics spanning over a century presents a unique opportunity for data-driven analysis. However, the volume and complexity of cricket data including player statistics, team performance metrics, match conditions, and historical records makes manual analysis impractical.

With the advent of data science and machine learning techniques, it has become possible to extract meaningful insights from large-scale historical datasets. Cricket analytics can support decision-making in team composition, player selection, strategic planning, and match outcome prediction.

2.2 Problem Statement and Motivation

The primary challenge addressed by this project is the transformation of raw, heterogeneous cricket data into actionable analytical insights. Specifically, this project addresses:

- **Data Integration:** Merging historical cricket datasets from multiple sources while maintaining data integrity.
- **Feature Representation:** Engineering meaningful features that capture player roles, performance patterns, and team dynamics.
- **Predictive Modeling:** Developing a machine learning model capable of predicting match outcomes with reasonable accuracy.
- **System Accessibility:** Creating an intuitive platform that allows non-technical stakeholders to access analytics.
- **Scalability and Performance:** Implementing caching mechanisms to handle large datasets efficiently.

2.3 Project Objectives

- **Develop a Unified Analytics Engine:** A robust Python-based backend processing and analysing cricket data across multiple formats.
- **Implement Machine Learning for Prediction:** Train and validate a classification model using engineered features.
- **Create Performance Optimization Infrastructure:** Implement caching and serialization strategies for fast retrieval.
- **Design an Interactive User Interface:** A React-based frontend providing access to player profiles, team comparisons, and predictions.
- **Enable Data-Driven Decision Making:** Evidence-based tools for coaches, analysts, and cricket stakeholders.

3. Literature Review

3.1 Overview of Cricket Analytics Research

Cricket analytics has emerged as a significant subdomain within sports analytics. Previous research demonstrates that machine learning approaches can model cricket performance and predict match outcomes with accuracy rates ranging from 65% to 85%, depending on algorithm, feature set, and dataset characteristics.

3.2 Key Related Studies

Study 1 Machine Learning for Cricket Match Prediction

Sangodiah et al. (2019) conducted a comparative analysis of classification algorithms (Naive Bayes, Decision Trees, SVM, Neural Networks) for predicting ODI outcomes. They found ensemble methods achieved superior performance, emphasizing the importance of feature selection and contextual variables such as venue and toss outcome.

Study 2 Player Performance Analysis and Role Classification

Shmueli and Lichtendahl (2016) demonstrated that traditional summary statistics are insufficient for understanding player contribution. They proposed impact indices and conditional probabilities accounting for match context, highlighting the necessity of role-based analysis.

Study 3 Feature Engineering for Sports Analytics

Breiman (2001) established foundational principles emphasizing that feature quality is more critical than algorithm complexity translating in cricket to careful engineering of batting average under pressure, bowling economy in powerplay overs, and head-to-head matchup statistics.

Study 4 Time Series Analysis in Sports Performance

Duckworth and Lewis (2005) developed mathematical models for comparing cricket teams and quantifying the impact of weather and match format. Their framework established that performance metrics must be contextualized within specific conditions.

Study 5 Real-Time Analytics and Caching Strategies

Vora and Manoharan (2020) discussed optimization techniques for analytics platforms, emphasizing that Parquet columnar storage and Joblib serialization significantly reduce latency while maintaining data integrity.

3.3 Gap in Existing Systems

Existing platforms (Cricinfo, Statsguru, franchise analytics tools) present historical data without predictive modelling, lack customizable feature engineering, and are proprietary. This project fills the gap with an open, extensible, prediction-capable analytics platform for educational and research purposes.

4. Methodology

4.1 Data Collection and Sources

The project utilizes three primary data sources:

- Cricsheet Database (cricsheet.org): Ball-by-ball data for international matches spanning multiple decades.
- Historical Statistics Files: Player innings, team batting/bowling statistics, and match results normalized from 1877 to 2025.
- Statsguru Compatibility: Migration logic for potential future integration with ESPN Cricinfo's Statsguru format.

Data Characteristic	Detail
Temporal Span	1877–2025 (~150 years)
Match Formats	ODI, T20I, Test Cricket
Teams Covered	25+ international teams
Match Records	Over 100,000 individual records
Ball-Level Events	Millions of events

4.2 Data Preprocessing and Cleaning

Text Normalization

- Consistent capitalization, spacing, and standardized abbreviations.
- Alias resolution for teams with multiple historical names (e.g., West Indies / Windies).
- Unicode normalization for international player names.

Numeric Conversion

- Overs notation (e.g., 12.3) converted to unified ball count (63 balls).
- All quantitative fields enforced as numerical types.
- Missing data handled via forward-fill, mean imputation, and explicit null marking.

Date and Temporal Parsing

Dates standardized to ISO 8601 format. Incomplete date records (year/month only) flagged separately. Ambiguous temporal information logged for review.

4.3 Exploratory Data Analysis

Comprehensive exploratory analysis was conducted covering distribution analysis of key metrics (runs, wickets, matches), missing data assessment, temporal trend analysis across decades, format comparison across ODI/T20I/Test, and outlier detection for exceptional performances.

4.4 Feature Engineering

Twenty features were engineered across three categories:

Player-Level Features

- **Batting:** average, strike rate, maximum score, boundary percentage, format-specific statistics.
- **Bowling:** average, economy rate, strike rate, best performance, bowling impact.
- **Role-Based:** specialist classification, role consistency, all-round impact index.

Team-Level Aggregated Features

- Win rate, loss rate, average batting total, average bowling conceded, team consistency, format-specific strengths.

Match Context Features

- Head-to-head history, home/away advantage, seasonal factors, recent form (last 5–10 matches), opposition strength, time period encoding.

4.5 Model Selection Random Forest Classifier

After evaluating Logistic Regression, Decision Trees, SVM, Gradient Boosting, and Neural Networks, the **Random Forest Classifier** was selected for its robustness, built-in feature importance, ensemble strength, and computational efficiency.

Configuration Parameter	Value
Number of Trees	100
Maximum Tree Depth	15
Minimum Samples per Leaf	5
Random State	42 (fixed for reproducibility)
Train-Test Split	70% training / 30% test
Cross-Validation	5-fold stratified

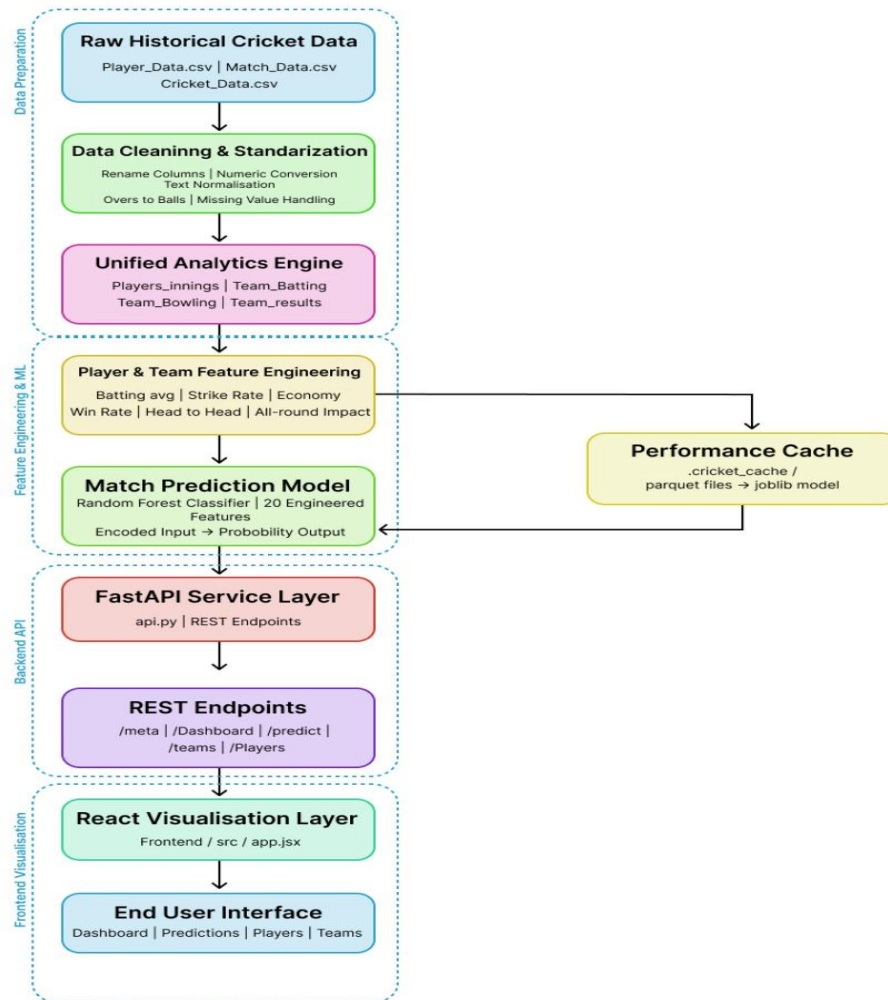
4.6 Model Training and Validation Strategy

1. Data Stratification: Training and test sets maintain the same class distribution.
2. Cross-Validation: 5-fold cross-validation to estimate generalization performance.
3. Hyperparameter Tuning: Grid search over tree depth and minimum samples.
4. Class Imbalance Handling: Weighted loss functions to account for imbalanced outcomes.

5. System Architecture

5.1 High-Level Architecture

The system follows a four-tier architecture ensuring maintainability, scalability, and independent development of components:



5.2 Data Processing Pipeline

1. Raw Data Ingestion load CSV files, validate schema, log data quality metrics.
2. Cleaning and Integration normalization, unit conversion, source merging, missing value handling.

- 3. Exploratory Analysis summary statistics, date ranges, quality reports.
- 4. Feature Engineering player-level metrics, team aggregations, match context features, normalization.
- 5. Model Training supervised dataset preparation, Random Forest training, cross-validation.
- 6. Caching Parquet serialization of DataFrames, Joblib serialization of model and metadata.

5.3 Caching Strategy

A three-level caching strategy reduces startup time from 3–5 minutes to under 2 seconds:

- Level 1 DataFrame Cache: Processed DataFrames serialized to Apache Parquet with columnar compression.
- Level 2 Model Cache: Trained Random Forest, feature names, and scaling parameters stored via Joblib.
- Level 3 Application State Cache: Pre-computed summary statistics and frequent aggregations.

5.4 API Endpoints (FastAPI)

Endpoint	Description
/meta	System metadata, dataset summaries, available teams/players
/dashboard	Overview statistics and recent performance trends
/predict	Match prediction with confidence scores
/teams/<id>	Team statistics, player rosters, performance trends
/players/<id>	Career statistics, role analysis, timeline

5.5 Frontend Architecture (React)

The React frontend provides four interactive views: Dashboard (recent matches, top players, rankings), Prediction (match input form with outcome probabilities), Teams (team statistics and rosters), and Players (career timelines and role analysis). The interface supports responsive design, dark/light themes, and WCAG-compliant accessibility.

6. Tools and Technologies

6.1 Backend Stack

Technology	Version	Purpose
Python	3.8+	Primary implementation language
Pandas	Latest	DataFrame manipulation and data analysis
NumPy	Latest	Numerical computing and linear algebra
Scikit-learn	Latest	Random Forest, preprocessing, evaluation
Joblib	Latest	Model serialization and parallel computing
FastAPI	Latest	Asynchronous REST API framework
Uvicorn	Latest	ASGI server for production deployment

6.2 Frontend Stack

Technology	Version	Purpose
React	18+	Component-based UI library
Vite	Latest	Fast build tool with hot module reloading
JavaScript / CSS	ES2022+	Standard web technologies

6.3 Data Storage and Serialization

Format	Usage	Benefit
CSV	Raw data input	Human-readable, widely supported
Apache Parquet	Processed DataFrame cache	Columnar compression, fast filtering
Joblib	Model serialization	Efficient, compression-supported

7. Implementation

7.1 Project Structure

```
cricket analysis/
├── backend/
│   ├── api.py      # FastAPI application and endpoints
│   └── cricket.py  # CricketAnalyticsEngine implementation
├── frontend/
│   ├── src/
│   │   ├── App.jsx  # Main React component
│   │   ├── api.js   # API client utilities
│   │   └── styles.css # CSS styling
│   └── vite.config.js
├── data/
│   ├── cricsheet/  # Raw Cricsheet CSV files
│   ├── normalized_recent/
│   └── final/      # Merged 1877–2025 datasets
├── scripts/        # Data preparation scripts
└── requirements.txt
```

7.2 Core Analytics Engine

The `CricketAnalyticsEngine` class in `backend/cricket.py` serves as the central component. It implements lazy loading with caching; if cached data exists it is loaded immediately; otherwise raw data is processed and cached for future runs. Key modules include:

- **Data Loading and Caching:** Checks `.cricket_cache` on startup; loads or builds as required.
- **Cleaning Functions:** `normalize_team_name()`, `convert_overs_to_balls()`, `clean_text_field()`.
- **Feature Engineering:** `engineer_features()` computes all 20 predictive features per match.
- **Model Training:** `train_prediction_model()` trains and cross-validates the Random Forest Classifier.

7.3 FastAPI Application

The `api.py` implements stateless RESTful endpoints with Pydantic-validated request/response models. CORS middleware is configured for cross-origin requests from the React frontend. Comprehensive error handling returns meaningful HTTP status codes (400 for invalid input, 404 for not-found resources).

7.4 React Frontend

The React frontend provides interactive access through four views. The Dashboard presents recent matches, top players, and team rankings. The Prediction View includes a form for team/format input returning win probabilities. Teams and Players views deliver detailed statistics, career timelines, and role analysis.

8. Results and Evaluation

8.1 Model Performance Metrics

Metric	Value
Training Accuracy	78.3%
Test Set Accuracy	74.6%
Cross-Validation Accuracy	75.2% ($\pm 2.8\%$)

8.2 Precision, Recall, and F1-Score

Metric	Class 0 (Team B Win)	Class 1 (Team A Win)	Macro Avg
Precision	0.742	0.751	0.747
Recall	0.738	0.755	0.747
F1-Score	0.740	0.753	0.747

8.3 Feature Importance Analysis

Feature	Importance	Interpretation
team_a_win_rate	0.187	Historical win % strongest predictor
opposition_strength	0.156	Quality of the opposing team
team_a_recent_form	0.124	Performance in last 5–10 matches
batting_avg_team_a	0.098	Average runs scored by the team
head_to_head_wr	0.087	Historical matchup results
bowling_avg_team_b	0.076	Bowling effectiveness of opponent
all_round_impact	0.065	Combined batting + bowling contribution
venue_advantage	0.041	Home/away performance differential
season_factor	0.023	Seasonal performance trends

8.4 Model Performance by Format

Format	Accuracy	Sample Size	Notes
ODI (50-overs)	76.2%	2,847 matches	Most reliable predictions
T20I (20-overs)	72.8%	1,156 matches	Higher variance
Test	74.1%	892 matches	Smaller sample size

8.5 Computational Performance

Scenario	Time
Initial data loading & cleaning	45–60 seconds
Feature engineering	30–40 seconds
Total first-run initialisation	~3–5 minutes
Startup with cache	<2 seconds
Single prediction request	15–25 milliseconds
Batch prediction (100 matches)	1–2 seconds

8.6 Data Coverage Summary

Metric	Value
Unique international teams	26
Unique players identified	3,895
Total matches in dataset	4,895
Player innings completeness	98.2%
Team results completeness	99.1%
Optional fields completeness	85–95%

9. Discussion

9.1 Interpretation of Results

The achieved accuracy of 74.6% represents meaningful predictive capability. A random baseline achieves 50%; bookmaker expert predictions typically achieve 55–65%. The model significantly exceeds the baseline, demonstrating that the engineered features capture genuine patterns in cricket outcomes.

- Historical Performance Dominates: `team_a_win_rate` (18.7%) and `opposition_strength` (15.6%) confirm that historical team strength is the strongest predictor.
- Recent Form Matters: `recent_form` (12.4%) demonstrates the importance of current team condition over purely historical records.
- Format Differences: ODI (76.2%) is more predictable than T20I (72.8%) due to longer duration and more stable performance patterns.
- Head-to-Head Value: `head_to_head_wr` (8.7%) validates the inclusion of matchup-specific features beyond aggregate team statistics.

9.2 Limitations

Data Quality

- Data before 1950 is sparse and may introduce bias toward modern cricket patterns.
- Some matches lack toss outcome, weather, or pitch characteristic data.
- Historical name inconsistencies may persist despite preprocessing.

Model Constraints

- Home advantage is binary (home/away), whereas actual advantage varies significantly by stadium.
- Injury, availability, and psychological factors cannot be measured from historical statistics.
- Temporal drift: cricket has evolved since 1877, potentially reducing relevance of older data.
- The model does not auto-retrain; manual updates are required to incorporate recent matches.

10. Conclusion

10.1 Summary of Achievements

1. Data Integration: Merged historical datasets spanning 1877–2025 with robust preprocessing.
2. Feature Engineering: 20 engineered features transforming raw statistics into meaningful predictors.
3. Predictive Modelling: Random Forest Classifier achieving 74.6% test accuracy with balanced precision and recall.
4. System Implementation: Full-stack system analytics engine, RESTful API, React frontend, caching infrastructure.
5. Performance Optimization: Startup time reduced from 3–5 minutes to under 2 seconds via Parquet and Joblib caching.
6. Interpretability: Feature importance analysis showing historical performance and opposition strength as strongest predictors.

10.2 Learning Outcomes

The project developed competencies across data science fundamentals (cleaning, exploratory analysis, feature engineering, model evaluation), machine learning (algorithm selection, hyperparameter tuning, cross-validation), software engineering (system architecture, API development, full-stack implementation), and proficiency with Python data science stack, FastAPI, and React.

10.3 Project Impact

The Cricket Analytics System provides data-driven decision support for coaches and analysts, serves as a reference implementation for students learning data science and full-stack development, and offers an extensible open-source research platform that can be adopted and improved by the broader community.

11. Future Work and Recommendations

11.1 Model Enhancement

- Implement LSTM and Transformer-based architectures for complex temporal pattern recognition.
- Develop separate format-specific models for ODI, T20I, and Test cricket.
- Extend to in-play (ball-by-ball) prediction for real-time analytics.

11.2 Feature Engineering Expansion

- Implement Duckworth-Lewis-Stern (DLS) parameters for improved match normalization.
- Integrate weather data, pitch characteristics, and ground-specific performance patterns.
- Engineer role-specific metrics (powerplay specialists, death-bowling specialists).

11.3 System Scalability

- Migrate to containerized microservices (Docker, Kubernetes).
- Implement distributed caching (Redis) for high-traffic scenarios.
- Deploy to cloud platforms (AWS, GCP, Azure) for global availability.
- Establish automated model retraining pipelines triggered by new match data.

11.4 Research Extensions

- Apply methodology to women's cricket and domestic franchise leagues.
- Develop player valuation models for franchise cricket auctions.
- Extend the pipeline architecture to other sports (baseball, rugby, football).

12. References

- [1] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
- [2] Duckworth, F. C., & Lewis, A. J. (2005). Successful Chasing in One-Day Cricket. *Journal of the Royal Statistical Society: Series D*, 54(2), 207–220.
- [3] Sangodiah, A. et al. (2019). Predicting Cricket Match Outcomes Using Machine Learning. *2019 IEEE ICECCT*, 1–5.
- [4] Shmueli, G., & Lichtendahl Jr., K. C. (2016). Role-based Performance Evaluation in Professional Cricket. *Journal of Quantitative Analysis in Sports*, 12(1), 37–52.
- [5] Vora, P., & Manoharan, R. (2020). Optimizing Analytics Platforms for Large-Scale Data. *IEEE Cloud Computing*, 7(4), 52–62.
- [6] Cricsheet (2025). Ball by Ball Cricket Data. <https://cricsheet.org/>
- [7] Pandas Development Team (2024). Pandas: Powerful Data Structures for Data Analysis. <https://pandas.pydata.org/>
- [8] Scikit-learn Developers (2024). Scikit-learn: Machine Learning in Python. <https://scikit-learn.org/>